IMPERIAL

Department of Computing
70015: Mathematics for Machine Learning
Lecturer: Matthew Wicker

Lecture 01: Problem Settings & Linear Models

1.1 What is Machine Learning?

The most natural question to ask at the beginning of a course that seeks to systematically study machine learning is: *what is machine learning?* Below, we offer two definitions for readers to consider:

- `Machine learning is the study of algorithms which automatically perform a task by processing an example dataset.`
- `Machine learning is a computational approach that allows systems to automatically learn and improve from experience without being explicitly programmed.`

A key difference between these to definitions is that the first was carefully crafted by the instructor of this course to serve our needs. The second was generated by a large language model (LLM).[1] In the first definition, we would like to particularly highlight the words `algorithm`, `task`, and `dataset`. Characterizing these words into mathematical objects that we can manipulate and study is the core objective of this class.

1.1.1 Datasets

There should be no doubt that the main catalyst of progress in machine learning is the wide availability of data. Data contains all the information we use to infer or learn good models and without it machine learning would not exist as we know it. But mathematically, what is data? For a single example, we denote the input or feature space as $\boldsymbol{x} \in \mathbb{R}^n$ and the outputs or label space as $\boldsymbol{y} \in \mathcal{Y} \subseteq \mathbb{R}^m$. We will inspect the exact forms and interpretations of x and y below, but first lets consider how we might characterize collections of inputs and outputs as mathematical objects. It is often the case that we will have many such input and output examples. Below we provide three perspectives to characterize these objects:

[1] One might say the definition is unnecessarily anthropomorphic with its use of the word "experience."

Set Perspective Perhaps the most natural starting point for thinking about a dataset is to use set theory. For this, inputs and outputs are grouped into a set of N 2-tuples: $\{(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)})\}_{i=1}^{N}$ we can then consider things like creating disjoint sets of data to train different models or to validate our models. But this object is not as easy to work with as others.

Probability Perspective One of the richest perspectives on our dataset is to view each 2-tuple $(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)})$ as a sample from a joint distribution $P(\mathbf{X}, \mathbf{Y})$ using this perspective, it becomes clear that one relationship we are very interested in is the probability of an output given that we have observed a particular input, denoted $P(\mathbf{Y} = \boldsymbol{y} | \mathbf{X} = \boldsymbol{x})$. We will also see how a probabilistic perspective gives a natural mathematical framing of many learning principles.

Linear Algebra Perspective Calling the following the *linear algebra perspective* is perhaps misleading as the form we present here is most commonly used in the field of statistics. Where we consider an input as a vector $\boldsymbol{x} \in \mathbb{R}^{n \times 1}$ we can collect all of these vectors into a matrix $\boldsymbol{X} \in \mathbb{R}^{m \times n}$ where each row is itself one of our input samples. Recall that by default vectors are columns so we have the vector:

$$\boldsymbol{x} = \begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_n \end{bmatrix}$$

and collected into the matrix we have:

$$\boldsymbol{X} = \begin{bmatrix} \boldsymbol{x}^{(1)\top} \\ \vdots \\ \boldsymbol{x}^{(m)\top} \end{bmatrix} = \begin{bmatrix} \boldsymbol{x}_1^{(1)}, & \boldsymbol{x}_2^{(1)}, & \dots, & \boldsymbol{x}_n^{(1)} \\ \vdots, & \vdots, & \ddots, & \vdots \\ \boldsymbol{x}_1^{(m)}, & \boldsymbol{x}_2^{(m)}, & \dots, & \boldsymbol{x}_n^{(m)} \end{bmatrix}$$

this matrix that collects all of our features (called explanatory variables in statistics) is then sometimes referred to as the the *design* matrix. The labels (or response variables in statistics) denoted $y \in \mathbb{R}$ are then collected into a vector $\boldsymbol{y}$.

We opt to describe this as the linear algebra perspective as there are many operations and tools from linear algebra that can help us understand our dataset once it is in this form. For example, consider the span of the column space of X, if it is less than n then we know that two of our columns are linear dependent and thus our data representation contains redundancy that we should remove to improve the computational complexity of our model operations. We will begin considering other similar operations in our next lecture.

1.1.2 Tasks

Over the past decades, numerous fascinating machine learning tasks have cropped up. As early as the 1990s, neural networks were used in autonomous driving. In 2012, the ImageNet challenge demonstrated the generalization power of neural networks in large-scale image classification. In

2016, AlphaGo beat the world champion in Go, and most recently in 2020 OpenAI released ChatGPT. Now, in 2025, we are seeing applications of AI in nearly all disciplines and being able to model and apply AI to novel settings is increasingly a valuable skill. But, to do so, we need to carefully understand the above advances, the techniques they leverage, and *why* they leverage them. That will be the focus of this course. Below, we begin to taxonomize a few key machine learning `tasks`.

Supervised Learning During the training of a model we always assume that we are able to observe the model's outputs. Supervised learning makes the additional assumption that we have observed the outputs according to the true process that we would like our model to mimic. As an example, we might have a set of images that contain cats and dogs. By labelling each image as either a cat or dog we can observe or supervise whether the outputs of our model match the outputs of the true labelling process.

Unsupervised Learning Unsupervised learning also known as *knowledge* discovery is when we have a set of feature vectors, but are unsure about their exact relation to one another. We may be interested for example, in what distribution the examples seem to be drawn from, or how many class labels may exist within these unlabelled examples.

Generative Learning A now extremely popular machine learning task is generative learning. The overall goal in this task is to take in a dataset (labelled or unlabelled) and predict or *generate* samples that have high probability of being from the same distribution as those in our dataset.

1.1.3 Algorithms

In our lecture slides I will briefly cover some algorithm differences; however, without getting into many technicalities it is difficult to convey meaningful algorithmic differences. Thus, we leave this section to be answered throughout the course. With that, we can dive into looking at our first machine learning model: the linear model.

1.2 A Practical View: Linear Models

When we choose a model in machine learning what we are explicitly doing is deciding how we think the inputs and outputs are related. It turns out that one of the simplest models one could chose can be made very powerful. The most basic linear model makes the assumption that we model the input-output relationship by pairing each dimension of a feature vector with an independent parameter. More formally, for each value $\boldsymbol{x}_i$ we assign a parameter θ_i and we take the sum of the product of each of these pairs to be our output. That is:

$$\hat{y} = \sum_{i=1}^{n} \boldsymbol{x}_i \theta_i.$$

Recall that this is precisely the summation notation for the dot product $\theta^\top \boldsymbol{x}$. One curiosity about this model that we will resolve in our next lecture is that $\boldsymbol{x} = \mathbf{0} \implies \hat{y} = 0$. Intuitively, this means that we can only model lines that cross through the origin.

1.2.1 Data Preprocessing

Understanding that many lines do not cross through the origin, one potential solution is to normalize our data such that we believe this is the case. There are many prospective ways of accomplishing this goal. For example, consider the idea of simply selecting any of our observed feature-label pairs $(\boldsymbol{x}^{(i)}, y^{(i)})$. We then subtract $\boldsymbol{x}^{(i)}$ from every feature and $y^{(i)}$ from every label.

If we have no noise or outliers in our data, then this seems to be a completely sensible thing to do as in this transformed space we will have the property $\boldsymbol{x} = \mathbf{0} \implies \hat{y} = 0$ is satisfied exactly at $(\boldsymbol{x}^{(i)}, y^{(i)})$ in the original space. But what about in the case that we do have noise or outliers? In this case perhaps rather than selecting an arbitrary pair $(\boldsymbol{x}^{(i)}, y^{(i)})$, we could normalize using the mean values:

$$\boldsymbol{x}^{\mu} = \frac{1}{n}\sum_{i=1}^{n} \boldsymbol{x}^{(i)}, \qquad \boldsymbol{y}^{\mu} = \frac{1}{n}\sum_{i=1}^{n} y^{(i)}$$

In this case, we still incur the error stemming from outliers, but we reduce the effect of these outliers by averaging them with the rest of our data. The above scheme is known as *centering* your data and we will see how useful this can be in future lectures.

1.2.2 Implementation Implications

Additionally, we should be careful and considerate about the computational implications of our model selection. When multiplying our data matrix by a parameter, $\boldsymbol{X}\theta$ what we really have is a matrix-vector product which along with matrix multiplications make up 99% of the floating point operations (flops) in modern models (Ivanov et al., 2021). In lecture, we will discuss a few of the implementation practicalities, we omit them in these lecture notes, however.

1.3 Linear Regression

We now turn to covering the basics of linear regression. While that may sound elementary, it is an important foundation to understand well as throughout this course we will use it to compare and contrast different learning approaches. Additionally, it is the most analytically tractable (meaning, can be dealt with in closed-form) of the models that we will learn this year so it is an important place to develop your intuition. A final preliminary note: while we will unpack the mathematics behind some modeling decisions in this lecture, many will remain *because I said so* – this is simply because the course has to start somewhere and we will justify and deepen our understanding of these decisions in future lectures.

Linear regression is exactly what the name suggests: performing regression with a linear model. However, in a class such as mathematics for machine learning, this is not quite rigorous enough. Linear regression is when we perform regression using a model whose inputs and outputs are related by a linear transformation.

Definition 1.3.1. Linear transformation A linear transformation between two vector spaces V and W is a map $T : V \to W$ such that the following hold:

- $T(v_1 + v_2) = T(v_1) + T(v_2)$ for any vectors v_1 and v_2 in V, and
- $T(\alpha v) = \alpha T(v)$ for any scalar α.

One can quickly check that the model we defined at the end of last lecture satisfies this definition, but what does it mean to *learn* in this model?

1.4 Learning as Optimization

Assuming domain of $\mathbb{R}^n$ and a one dimensional co-domain (e.g., $y \in \mathbb{R}$) we can write our model as $f(x) = \boldsymbol{x}^\top \theta$. Thus our model can be written as:

$$\hat{y}^{(i)} = \boldsymbol{x}^{(i)\top} \theta$$

The key idea behind learning is to find θ such that $\hat{y}^{(i)} \approx y^{(i)}$. But, as stated it is unclear what $\approx$ means exactly. For each learning problems it will take on a different meaning. For example, if you predict that my yearly salary is 41,085 pounds and it is actually 42,085 pounds you were off by 1000. If you were age is 1028 you would also be off by 1000, but there is no sense in which your prediction is approximately equal to my age. This provides yet another motivation for standardization of our data.

Standardisation In the above example we are thinking about the critical question of what we mean for predictions to be *close* to the true output. But the issue we observe can be solved by another form of data preprocessing (recall, in last lecture we saw centering). Here, we want to standardize our response variables to be between -1 and 1. This allows us to compare the performance of a model/algorithm without having to reason about data-specific average error values.

1.4.1 Norms as Error Models

Given that we have standardized the outputs of our datasets, we still would like to measure how *close* our prediction is to the desired output. One of the most common ways to measure a model's predictive error is to use *norms*. The most common norm student's will have seen to this point is the Euclidean norm:

Definition 1.4.1. Euclidean Norm Given a vector $\boldsymbol{x} \in \mathbb{R}^n$ the Euclidian norm of the vector, denoted $||\boldsymbol{x}||_2$ is given by:

$$||\boldsymbol{x}||_2 = \sqrt{\sum_{i=1}^{n} \boldsymbol{x}_i^2}$$

The Euclidean norm has the convenient geometric intuition of being the straight-line distance between the origin $\boldsymbol{x}$. Similarly, the straight-line distance between $\boldsymbol{x}$ and $\boldsymbol{y}$ (given both are in $\mathbb{R}^n$) is $||\boldsymbol{x} - \boldsymbol{y}||_2$. As a measure of error, we might consider $||\boldsymbol{y} - \hat{\boldsymbol{y}}||_2$ which is the length of the shortest straight-line between our prediction and the true value we want.

Definition 1.4.2. ℓ_p **Norm** Given a vector $\boldsymbol{x} \in \mathbb{R}^n$ the ℓ_p norm of the vector, denoted $||\boldsymbol{x}||_p$ is given by:

$$||\boldsymbol{x}||_p = \sqrt[p]{\sum_{i=1}^{n} \boldsymbol{x}_i^p}$$

It is clear to see that the ℓ_2 norm is the same as the Euclidean norm. Two special cases of this definition exist: ℓ_0 and ℓ_∞. The ℓ_0 norm is defined as the number of non-zero elements of the vector. The ℓ_∞ norm on the other hand is defined as the largest element of the vector.

1.4.2 Best model is given by the argmax parameter

Let us now collect all of our model predictions into a vector, $\hat{\boldsymbol{y}}_i := \theta^\top \boldsymbol{x}^{(i)}$ and collect the true values into a corresponding vector $\boldsymbol{y}$. Combining these vectors with ℓ_p norms above we can recover several classical error functions. Notably the Root Mean Squared Error (RMSE): $||\hat{\boldsymbol{y}} - \boldsymbol{y}||_2$ and the Mean Squared Error (MSE) $||\hat{\boldsymbol{y}} - \boldsymbol{y}||_2^2$.

By rewriting $\hat{\boldsymbol{y}}$ in terms of our parameters, and by noticing that both $\boldsymbol{X}$ and $\boldsymbol{y}$ are fixed, we can express the above errors as functions of the parameter value which we will call the *loss*:

$$\mathcal{L}(\theta) = ||\boldsymbol{X}\theta - \boldsymbol{y}||_2^2$$

It then becomes very natural to consider two values for the model parameters θ' and θ'' and ask which one is *better* according to the loss. It is clear that the value with the lower loss is the better parameter value as it better reproduces the labels/responses we expected from our model. Thus for any two arbitrary parameters θ' and θ'' we say that θ' is a better parameter if $\mathcal{L}(\theta') < \mathcal{L}(\theta'')$.

Finally, we may be interested in finding the *best* parameter setting. Specifically, we might be interested in finding a parameter $\theta^\star$ such that $\forall \theta' \neq \theta^\star, \mathcal{L}(\theta') \geq \mathcal{L}(\theta^\star)$. Notice, that this happens when the value of $\mathcal{L}$ is at its minimum (and assumes the minimum is unique). However, we do not care about *what* the minimum value is; rather we care about *where* it achieves this minimum. For this we introduce the argmin function which returns the *argument where the function achieves its minimum*. We therefore say that:

$$\theta^\star = \underset{\theta \in \mathbb{R}^n}{\operatorname{argmin}} \mathcal{L}(\theta)$$

Bibliography

Ivanov, A., Dryden, N., Ben-Nun, T., Li, S., and Hoefler, T. (2021). Data movement is all you need: A case study on optimizing transformers. *Proceedings of Machine Learning and Systems*, 3:711–732.

Wang, L., Zhang, X., Su, H., and Zhu, J. (2024). A comprehensive survey of continual learning: theory, method and application. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.